PASTA Threat Model: Sneaker Marketplace App

Fictional scenario. Threat model built using the Process for Attack Simulation and Threat Analysis (PASTA) framework, completed as part of the Google Cybersecurity Professional Certificate.

Report date: 22 March 2026

Prepared by: Security Team | Prepared for: Engineering Leadership & Product Owner | Classification: Internal

A sneaker resale and collector company is preparing to launch a mobile marketplace app connecting buyers and sellers. This threat model works through all seven PASTA stages to evaluate whether the app is ready to launch from a security standpoint, and what needs to change before it is.

Executive Summary

This assessment evaluates the sneaker marketplace app's readiness for launch using the PASTA threat modeling framework. Two vulnerabilities were identified that are directly exploitable given the app's current architecture and sit on the path to its most sensitive data (customer accounts and payment information): unparameterized SQL queries in the search/listing feature, and weak session token handling in account authentication. Both carry a Critical risk rating and are recommended as launch blockers, with an estimated low-to-moderate engineering lift to remediate (see Stage VII). Two additional findings, missing multi-factor authentication and unmanaged API credentials, carry High and Medium risk ratings respectively and are recommended for remediation before or shortly after launch. No Critical or High findings were identified outside the authentication and data-access layers.

Recommendation: Do not launch until the two Critical findings are remediated.

Scope

In scope: The mobile app's account authentication, product search/listing, buyer-seller messaging, and payment processing flows, and the technologies supporting them (API layer, PKI/encryption, password hashing, SQL database).

Out of scope: The payment gateway provider's own internal security posture (assessed separately under vendor due diligence), physical security of hosting infrastructure, and mobile OS-level or device-level vulnerabilities. These should be addressed through separate assessments.

Assumptions

This assessment starts from a brief, high-level business description (core features, a five-item technology list, and a single sample process). As in most real threat modeling engagements, the initial brief did not fully specify the application's architecture, so the following were inferred to build a complete model, consistent with the stated business objectives and technologies:

- **Data stores:** A separate credentials store (D1), product/inventory store (D2), and transaction store (D3) are assumed, since the brief describes account management, listings, and payments as distinct features, but does not specify the underlying data architecture.

- **External payment gateway:** The brief states the app must offer "several payment options" and handle payment properly to avoid legal issues; a third-party payment gateway API is assumed as the mechanism, consistent with standard practice and the app's listed use of RSA key exchange.

- **Process boundaries:** The four processes in Stage III (authentication, search/listing, messaging, payment) are derived from the features described in Stage I; the course materials provided only a single sample process (search) as a starting point.

These assumptions are reasonable for a consumer marketplace app of this kind, but in a real engagement they would be confirmed with the development team during Stage II, as noted in the course guidance.

Stage I: Define Business & Security Objectives

The app must process financial transactions directly between buyers and sellers, **which means payment card data is in scope from day one and PCI-DSS compliance is a business requirement, not an optional hardening step.**

Users can create accounts and manage profiles, **and the business has explicitly flagged data privacy as a customer trust concern, meaning account and profile data needs protection commensurate with a consumer-facing financial app, not a basic marketplace listing site.**

Buyers and sellers communicate directly through in-app messaging and ratings, **introducing a persistent, authenticated session that needs to remain secure across potentially long user interactions, not just during checkout.**

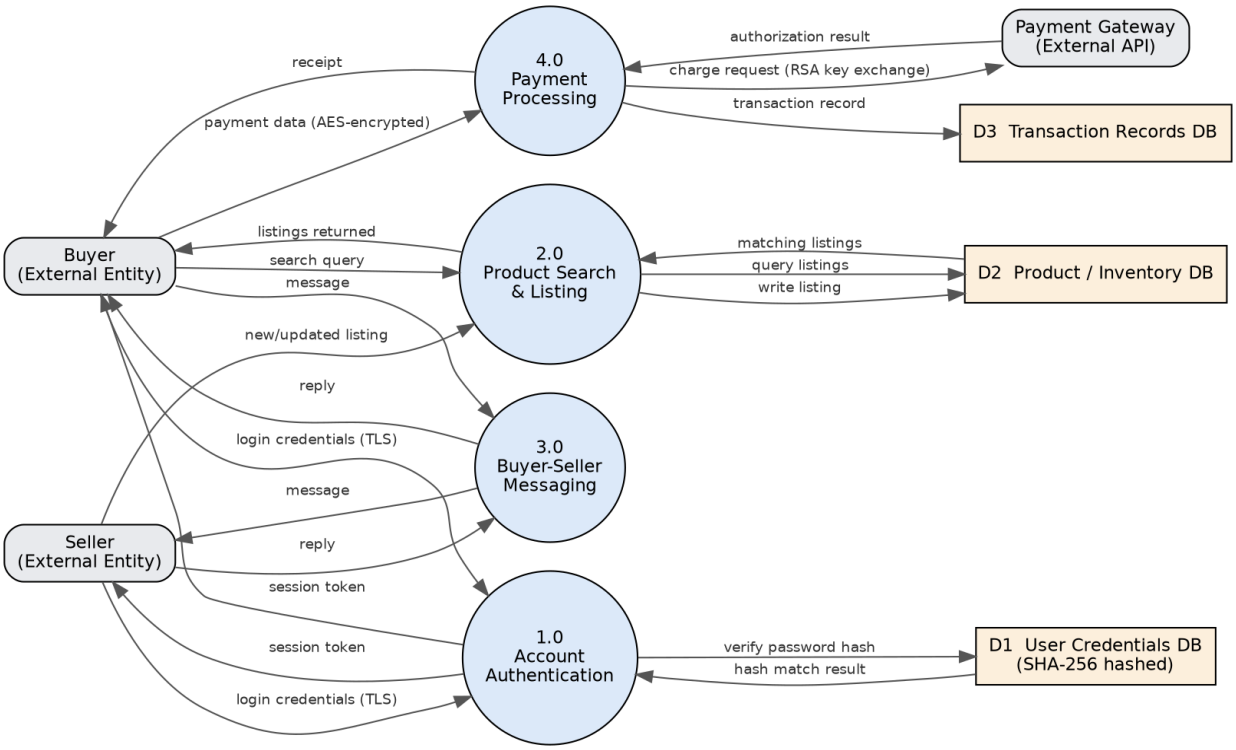
Stage II: Define the Technical Scope

Technology	Role in the App	Priority
API (third-party integrations)	Connects the app to external services, most notably the payment gateway, and exchanges data between buyers, sellers, and internal systems.	1st
PKI (AES + RSA)	Encrypts sensitive data (AES) and manages secure key exchange between the app and user devices (RSA), primarily during payment and authentication.	2nd
SHA-256	Hashes passwords and credit card numbers so sensitive values are never stored or compared in plaintext.	3rd
SQL	Stores and retrieves listing, inventory, and transaction data; directly reachable from user-facing search and listing forms.	4th

The API layer is prioritized first because it is the app's largest attack surface: it connects buyers, sellers, and the external payment gateway, and any weakness here (a leaked token, an unauthenticated endpoint, insufficient rate limiting) exposes every other technology behind it, including the encryption and hashing layers, which only protect data that has already reached a trusted system. PKI is reviewed second because it directly protects payment data in transit and is a PCI-DSS requirement, followed by SHA-256, which protects data at rest, and SQL, which is significant but sits behind the API and authentication layers rather than being directly internet-facing.

Stage III: Decompose the Application

The diagram below expands the single sample process (product search) provided in the course materials into a fuller data flow diagram covering account authentication, product search and listing, buyer-seller messaging, and payment processing, the four core process areas identified in Stage I and II.



Two details stand out from this diagram. First, the payment processing flow (4.0) is the only process that reaches an external system (the payment gateway), which makes it the highest-value target for interception and the reason PKI was prioritized in Stage II. Second, account authentication (1.0) gates access to every other process, since both search/listing and messaging assume a valid session token, meaning a weakness in authentication has consequences well beyond the login screen itself.

Stage IV: Threat Analysis

Injection attacks: The product search and listing processes (2.0) construct SQL queries directly from user-supplied input (search terms, new listing details). An external attacker could submit crafted input designed to manipulate these queries, potentially reading or modifying data beyond what the search or listing feature was intended to expose.

Session hijacking: Because a single session token from account authentication (1.0) grants access to messaging, listings, and purchase actions, an attacker who intercepts or predicts a valid session token could impersonate a buyer or seller for the lifetime of that session, without ever needing their password.

Stage V: Vulnerability Analysis

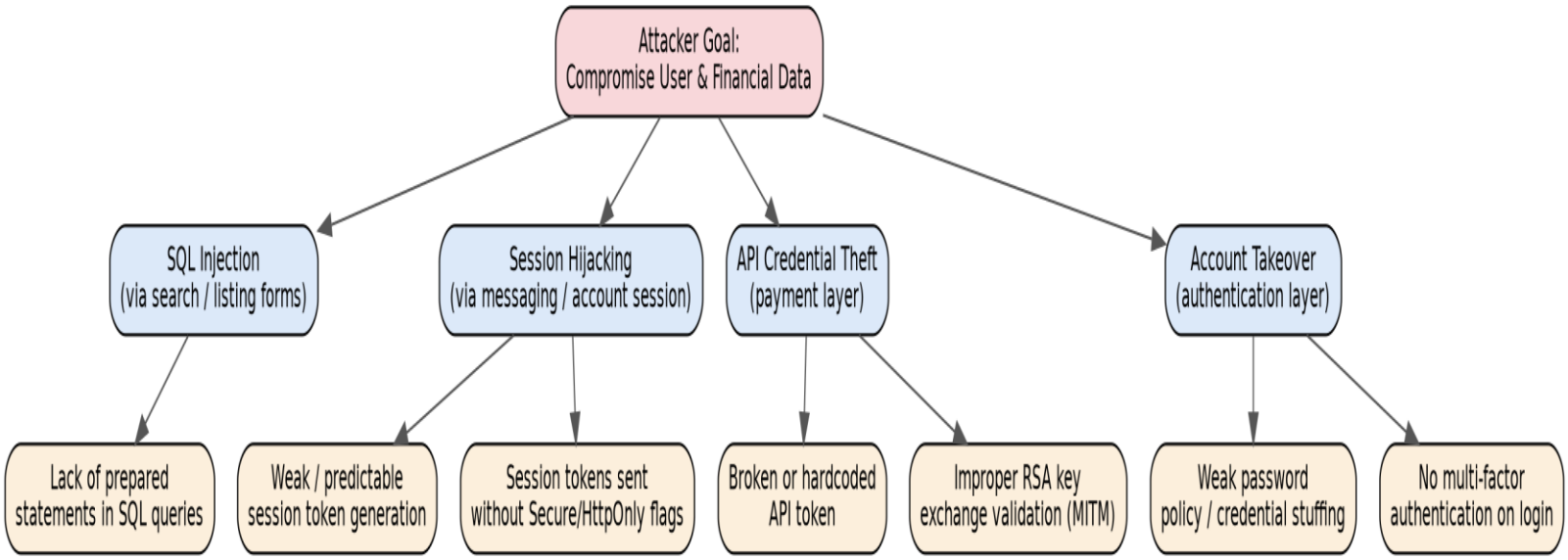
Lack of prepared statements — Risk: Critical (CWE-89, OWASP A03:2021 – Injection): If the SQL queries backing the search and listing processes concatenate user input directly into query strings instead of using parameterized queries, the injection threat identified above becomes directly exploitable

rather than theoretical. Rated Critical because it is reachable by any unauthenticated user through the public-facing search feature and can expose or modify data beyond the search's intended scope.

Weak or predictable session tokens — Risk: Critical (CWE-384, OWASP A07:2021 – Identification and Authentication Failures): If session tokens are generated with insufficient randomness, or transmitted without Secure and HttpOnly cookie flags, they become guessable or interceptable, directly enabling the session hijacking threat identified above. Rated Critical because a single compromised token grants access to messaging, listings, and purchase actions for the token's full lifetime.

Stage VI: Attack Modeling

The attack tree below expands the two threats and vulnerabilities identified above into a fuller picture of how an attacker could pursue the overall goal of compromising user and financial data, adding two additional branches (API credential theft and account takeover) that follow directly from the technologies prioritized in Stage II.



The tree illustrates that this app has more than one viable path to the same high-value target (user and financial data), which is typical for an application handling both authentication and payment. The SQL injection and session hijacking branches are rated Critical (see Stage V); the API credential theft and account takeover branches are rated Medium and High respectively, reflecting their comparatively higher barrier to exploitation. This matters for prioritization: a single fix, such as adding prepared statements, closes one Critical branch but leaves the others open, which is why Stage VII proposes controls addressing each branch rather than a single point solution.

Stage VII: Risk Analysis & Impact / Recommended Controls

The table below maps each finding to a specific control, an owner, and a timeline, ordered by risk rating and launch impact.

Finding / Risk	Recommended Control	Effort	Owner	Timeline

SQL injection (Critical)	Replace concatenated queries with parameterized queries / prepared statements across all search and listing endpoints	Low (isolated to data-access layer)	Backend Engineering	Before launch
Session hijacking (Critical)	Generate tokens via a cryptographically secure random source; enforce Secure and HttpOnly cookie flags on all session cookies	Low-Moderate	Backend Engineering	Before launch
Account takeover (High)	Add multi-factor authentication (MFA) at login	Moderate (requires SMS/email provider integration)	Product + Engineering	Before launch, or within 30 days if launch cannot be delayed
API credential theft (Medium)	Move API credentials to a managed secrets store; rotate tokens; scope each token to least privilege	Moderate	Platform/DevOps	Within 60 days post-launch

Recommendation: the application should not launch until the two Critical findings (SQL injection, session token handling) are remediated, since both are directly exploitable given the app's current architecture, both are reachable by an unauthenticated attacker, and both sit on the path to the app's most sensitive data. Both are also low engineering lift relative to their risk reduction, making them reasonable to require as launch gates rather than fast-follow items. MFA is strongly recommended before launch but could follow within 30 days if a hard launch date requires it; API token hardening carries the lowest immediate risk of the four findings and is reasonable to schedule shortly after launch.